AI-Powered Brute Force Detection in SIEM

MSc Cybersecurity and Digital Forensics project (University of Wales Trinity Saint David).

This project trains machine learning models to detect brute force attacks (SSH/FTP), tests them on real captured traffic, and connects a detection script to a live **Wazuh SIEM** watching a **Kali Linux** machine.

What this project does

1. Trains and compares 6 ML algorithms on the CICIDS 2017 dataset.
2. Compares the best model (Random Forest) against a simple rule-based detector.
3. Captures real traffic with Wireshark, turns it into flows with NFStream, and re-tests the models on it.
4. Removes the port number as a feature, so the model learns behaviour instead of just "port 22 or 21".
5. Reads Wazuh alerts and runs two detectors side by side: the ML model and a simple behavioural rule.

Results

On CICIDS 2017 (Tuesday file: 445,909 rows, about 3.1% attacks):

Method	F1-score	False positives
Random Forest	99.96%	0
Rule-based (Flow Packets/s > 100)	5.76%	44,278

On my own captured traffic (792 flows: 275 normal, 517 attack):

- Port number was the top feature (25.7% importance) in the first model.
- After removing it and adding behavioural features, `src_ip_frequency` became the top feature.

The traffic I captured is simulated, not a real attack. See Limitations.

Algorithms compared

Random Forest, XGBoost, Decision Tree, K-Nearest Neighbours, Logistic Regression, Linear SVM. All use the same 80/20 stratified split (`random_state=42`).

Files

File	What it is
`CICIDS2017.ipynb`	Trains and compares the 6 models, rule-based comparison, charts
`Realtime.ipynb`	Real traffic to flows (NFStream), behavioural features, model retraining, saves the model
`fake_server.py`	Simple fake server on port 2222 that logs connection attempts
`attacker.py`	Makes 500 rapid connection attempts to the fake server (simulated attack)
`wazuh_detector.py`	Reads Wazuh `alerts.json` and shows ML verdict and behavioural verdict per source IP
`bruteforce_model.pkl`	Saved Random Forest (behavioural features, no port)
`feature_names.pkl`	Feature names the model expects
`Attacker-Machine.png`, `Wazuh-Server.png`	Screenshots of the lab

Dataset (not included)

The dataset is too large for GitHub. Download `Tuesday-WorkingHours.pcap_ISCX.csv` from the CICIDS 2017 page of the Canadian Institute for Cybersecurity (University of New Brunswick), and place it where the notebook expects it. The `.pcap` files I captured are also not included.

Lab setup

- Wazuh 4.9.2 server on an Ubuntu VM (VirtualBox, bridged network)
- Kali Linux VM running the Wazuh agent (the monitored machine)
- The agent reads SSH logs through journald, because modern Kali has no `/var/log/auth.log`. Config used:

```xml
```

```
<localfile>
  <log_format>journald</log_format>
  <location>journald</location>
</localfile>
```

How to run

Notebooks: open `CICIDS2017.ipynb` and `Realtime.ipynb` in Google Colab. Mount your Drive and put the dataset and pcap files there.

Simulated attack:

```
python fake_server.py      # terminal 1
python attacker.py         # terminal 2
```

Detector (on the Wazuh server):

```
pip install joblib scikit-learn pandas
sudo python3 wazuh_detector.py
```

Place `bruteforce_model.pkl` and `feature_names.pkl` in the same folder. The script reads `/var/ossec/logs/alerts/alerts.json` directly (the Wazuh API gave me persistent 401 errors, so I did not use it).

Limitations

- **The ML model does not work well on Wazuh SSH logs.** It was trained on network-flow features (packet counts, bytes, durations). Wazuh SSH logs contain login events, not flows. In my tests the ML model said "Normal" while the behavioural rule (3 or more failed logins from one IP) correctly flagged the attack. This is a domain mismatch, and it is the main finding of the integration.
- The behavioural rule is simple. Wazuh already has its own SSH brute force rules, so this rule adds little. The value of the ML approach would show only if it were trained on features that exist in the log data.
- The captured attack traffic is simulated with Python sockets, not a real attack tool, so the model results on it are optimistic.
- CICIDS 2017 data is from a lab and may not match real networks. I used only one day of it.
- The test on my own traffic is small (792 flows).

Tools

Python, scikit-learn, XGBoost, pandas, NFStream, joblib, Wireshark, Wazuh, Kali Linux, VirtualBox, Google Colab.

Reference

Sharafaldin, I., Lashkari, A. H. and Ghorbani, A. A. (2018) "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization", ICISSP 2018.